AYUDANTÍA

Koa, CRUDs y Sequelize

Diseño de endpoints RESTful, manipulación del contexto HTTP y persistencia relacional sobre PostgreSQL.



SERVIDOR WEB KOA

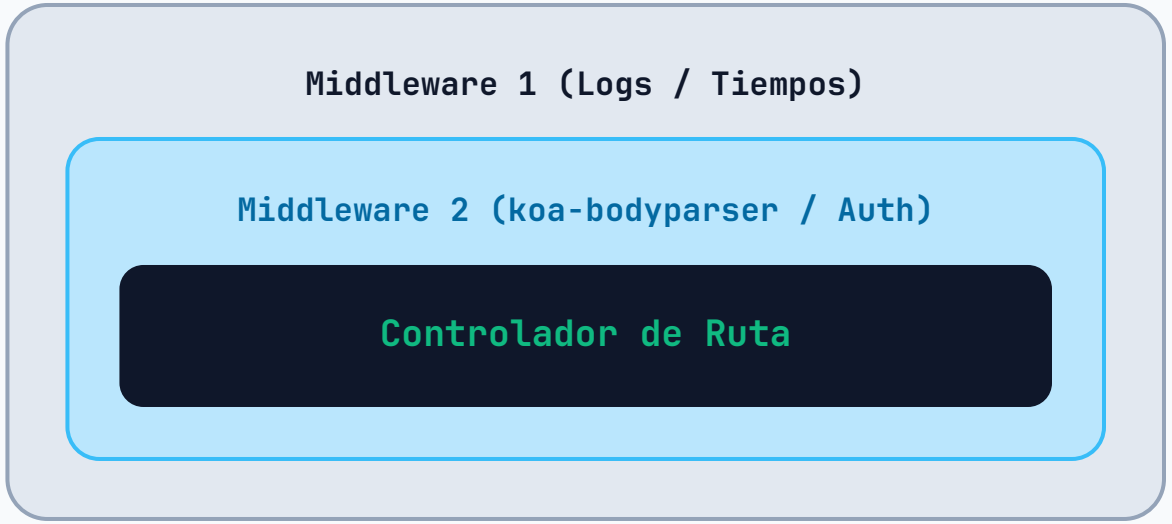
Koa: núcleo ligero y modelo de cebolla

Función intermedia: Ejecuta lógica entre la petición y la respuesta.

Tareas transversales: Registro de logs, validación y autenticación.

Flujo de ida: La solicitud atraviesa cada capa hacia el controlador.

Flujo de vuelta: Retoma el código posterior a await next().



CONTEXTO DE EJECUCIÓN

Anatomía de ctx y la función next()

ctx encapsula solicitud y respuesta HTTP. await next() cede el control al siguiente middleware.

`ctx.request`

Información de la petición HTTP recibida.

`ctx.response`

Objeto que prepara cabeceras de salida.

`ctx.body`

Carga útil serializada a formato JSON.

`ctx.status`

Código numérico del resultado HTTP.

CAPTURA DE PARÁMETROS

Entrada de datos: ¿Dónde buscar en ctx?

`/items/:id`

Route Params

`ctx.params`

Individualiza un recurso específico en la ruta.

`/items?sort=asc`

Query Params

`ctx.query`

Parámetros opcionales de filtro u orden.

`POST /items`

Body JSON

`ctx.request.body`

Carga útil leída mediante koa-bodyparser.

ARQUITECTURA RESTFUL

Identidad de una operación en APIs REST

Naturaleza Stateless

El servidor no guarda estado del cliente entre llamadas. Cada petición contiene toda la información necesaria para procesarse.

Identidad Única = Método HTTP + Ruta URL

`GET /cursos`

Obtiene la lista completa de cursos registrados.

`DELETE /cursos/123`

Elimina el curso con identificador numérico 123.

CONTRATO DE ENDPOINTS

Matriz de operaciones CRUD sobre /cursos

Operación	Verbo	Ruta	Propósito técnico	Idempotencia
Crear	`POST`	`/cursos`	Inserta un curso nuevo	No idempotente
Listar	`GET`	`/cursos`	Recupera la colección completa	Idempotente
Detalle	`GET`	`/cursos/:id`	Obtiene un curso según su id	Idempotente
Reemplazar	`PUT`	`/cursos/:id`	Sobrescribe el recurso completo	Idempotente
Modificar	`PATCH`	`/cursos/:id`	Edita campos específicos	No garantizada
Eliminar	`DELETE`	`/cursos/:id`	Remueve el registro de la base	Idempotente

SEMÁNTICA DE RESPUESTAS

Códigos de estado HTTP en un CRUD

200 OK: Operación exitosa con retorno de datos en consultas o ediciones.

201 Created: Creación confirmada de un nuevo recurso tras ejecutar POST.

400 Bad Request: Datos de entrada incompletos o JSON malformado.

404 Not Found: Recurso inexistente; retornar {"error": "Recurso no encontrado"}.

500 Internal Server Error: Falla no controlada en servidor o base de datos.

PERSISTENCIA DE DATOS

ORM: Mapeo Objeto-Relacional con Sequelize

Permite interactuar con la base de datos relacional usando métodos de programación orientada a objetos en lugar de redactar consultas SQL manuales.



INICIALIZACIÓN DEL ESQUEMA

Ciclo de vida del modelo: define() vs sync()

ETAPA 1 · MEMORIA DEL ORM

`sequelize.define()`

Estructura: Declara atributos y tipos de datos.

Ubicación: Ocurre en la memoria de la aplicación.

Momento: Antes de interactuar con la base de datos.

ETAPA 2 · MOTOR POSTGRESQL

`sequelize.sync()`

Conexión: Sincroniza el esquema con PostgreSQL.

Creación: Genera la tabla física si no existe.

Asincronía: Retorna una promesa al iniciar el servidor.

MÉTODOS DE CONSULTA

Manipulación de registros en Sequelize

INSERT INTO

create()

Inserta una nueva fila de datos en la tabla.

SELECT * LIMIT 1

findOne()

Recupera una única fila según la condición dada.

SELECT *

findAll()

Retorna todas las filas como arreglo de instancias.

CONTROL DE ESQUEMAS

Migraciones frente a Seeders en Sequelize

Migraciones

Propósito: Alterar el esquema de forma controlada y reversible.

Alcance: Estructura de tablas, columnas y restricciones.

Caso de uso: Agregar una columna nueva sin perder datos.

Seeders

Propósito: Poblar las tablas con registros iniciales o de prueba.

Alcance: Contenido de filas dentro de tablas ya creadas.

Caso de uso: Levantar el entorno local con cursos de ejemplo.

DISEÑO DE SOFTWARE

Separación de responsabilidades: Tres capas

ORIGEN

Cliente HTTP

Envía request JSON.

→

CAPA 1

Controlador

Gestiona ctx y rutas.

→

CAPA 2

Servicio

Valida reglas de negocio.

→

CAPA 3

Modelo ORM

Ejecuta persistencia.

Retorno inverso: PostgreSQL devuelve filas → Modelo instancia objetos → Controlador asigna ctx.status y ctx.body.

IMPLEMENTACIÓN EN KOA

Código: Del endpoint a la base de datos

```
// 1. Listar todos los cursos

router.get('/cursos', async (ctx) ⇒ {
     const cursos = await Curso.findAll();
     ctx.status = 200; ctx.body = cursos;
});

// 2. Crear un nuevo curso

router.post('/cursos', async (ctx) ⇒ {
     const { nombre, sigla } = ctx.request.body;
     const nuevo = await Curso.create({ nombre, sigla });
     ctx.status = 201; ctx.body = nuevo;
});
```

Asincronía: Uso obligatorio de async/await al consultar Sequelize.

Desestructuración: Extracción limpia desde ctx.request.body.

Estado explícito: Asignación de 200 o 201 antes de retornar.

SÍNTESIS TÉCNICA

Reglas de oro antes de programar

01 · IDENTIDAD REST

Definir cada operación con el par método HTTP y ruta.

02 · BODY PARSER

Registrar koa-bodyparser antes de leer ctx.request.body.

03 · CONTROL DE 404

Responder siempre con estado 404 y objeto JSON de error.

04 · USO DEL ORM

Encapsular todas las consultas en el modelo Sequelize.

ACTIVIDAD

Construcción de API Cursos

MODELO SEQUELIZE

Curso

id: INTEGER (PK, Auto)

nombre: STRING (Not Null)

sigla: STRING (Not Null)

Endpoint	Respuesta esperada
`POST /cursos`	201 Created con el objeto creado
`GET /cursos`	200 OK con listado vía findAll()
`GET /cursos/:id`	200 OK o 404 con JSON de error
`PUT /cursos/:id`	200 OK con datos actualizados
`DELETE /cursos/:id`	200 OK tras eliminar registro